INDIAN INSTITUTE OF INFORMATION TECHNOLOGY & COMPUTER SCIENCE

Summer Internship Programme 2026 — Track II

AI-POWERED PHISHING DETECTION

Evaluating Parametric and Non-Parametric Text Classification Methods

PROJECT 2 — FINAL DOCUMENTATION REPORT

2026

Author / Student	**Vishnu Thangaraj**
Enrollment ID	845488
Project Type	Machine Learning & Cybersecurity
Contact Email	vishnuthangaraj@proton.me
Submission Date	July 2026

Prepared as partial fulfillment of the requirements for the
IICT Summer Internship (Cyber & AI Track)

EXECUTIVE SUMMARY

ABSTRACT

Phishing remains one of the most prominent cybersecurity threats, leveraging social engineering and deceptive communication to compromise sensitive personal and organisational data. Traditional rule-based filtering systems are increasingly inadequate against zero-day phishing campaigns that dynamically alter their payload signatures and linguistic structures.

This project presents an empirical evaluation of machine learning models for detecting phishing emails using Natural Language Processing (NLP). We develop a synthetic corpus of legitimate and phishing emails, and apply Term Frequency-Inverse Document Frequency (TF-IDF) vectorisation to transform raw text into numerical feature representations.

Four distinct algorithms are trained and evaluated: Logistic Regression, Random Forest, Multinomial Naive Bayes, and a Multi-Layer Perceptron (MLP) Neural Network. Performance is measured across Accuracy, Precision, Recall, and F1-Score. Feature importance analysis is conducted to interpret model decisions. The results indicate that ensemble methods (Random Forest) and Neural Networks offer superior detection capabilities compared to linear models on this text-based dataset.

Keywords

Phishing Detection, Cybersecurity, Machine Learning, Natural Language Processing (NLP), TF-IDF, Text Classification

TABLE OF CONTENTS

1	Introduction & Background	4
2	Literature Review	6
3	Proposed Methodology	8
4	Dataset Description	9
5	Data Preprocessing	11
6	Feature Engineering (TF-IDF)	12
7	Machine Learning Models	13
8	Model Training Pipeline	15
9	Results & Evaluation	16
10	Feature Importance & Analysis	19
11	Prediction Module Inference	20
12	Advantages & Limitations	21
13	Conclusion & Future Scope	22
A	Appendix A: Python Source Code Part 1	23
B	Appendix B & C: Python Source Code Part 2 & Hyperparameters	24
D	Appendix D & E: Glossary	25

Chapter 1

1. INTRODUCTION

1.1 Background

Phishing attacks represent a pervasive social engineering technique where attackers masquerade as trusted entities to deceive victims into revealing sensitive information, such as login credentials, credit card numbers, or proprietary corporate data. In the digital era, email remains the primary vector for such attacks due to its ubiquity and the low cost of mass communication.

Traditional anti-phishing mechanisms have largely relied on blacklists and heuristic rule-based systems. While effective against known threats, these methods struggle with the dynamic and polymorphic nature of modern phishing campaigns. Attackers frequently alter sender addresses, domain names, and email content to evade signature-based detection.

Consequently, the cybersecurity industry is increasingly turning to artificial intelligence and machine learning (ML) to provide robust, adaptable defenses. By analyzing the structural, linguistic, and contextual features of emails, ML models can identify anomalous patterns indicative of phishing, even in zero-day attacks.

1.2 Impact of Phishing Attacks

The financial and reputational damage caused by phishing is immense. Organizations face direct monetary losses, legal liabilities, regulatory fines, and a loss of consumer trust following a data breach. Furthermore, phishing often serves as the initial access vector for more devastating attacks, such as ransomware deployment or advanced persistent threats (APTs).

1.3 Problem Statement

Despite advancements in email security, accurately distinguishing between legitimate and phishing emails remains challenging. Legitimate marketing and transactional emails often share characteristics with phishing attempts, such as urgent language or embedded links. Therefore, a highly precise classification system is required to minimize false positives (which disrupt normal business operations) while maximizing the detection of actual threats.

This project addresses the problem of automated phishing email detection by treating it as a binary text classification task. We aim to evaluate the efficacy of different machine learning algorithms in parsing and classifying email content.

1.4 Objectives

The primary objectives of this project are:

- To construct a representative synthetic dataset of legitimate and phishing emails.
- To implement a robust Natural Language Processing (NLP) pipeline for cleaning and vectorizing textual data.
- To train and optimize multiple ML models, including Logistic Regression, Random Forest, Naive Bayes, and a Neural Network.
- To evaluate these models using comprehensive metrics (Accuracy, Precision, Recall, F1-Score) and identify the most effective approach.
- To analyze model decision-making by extracting and visualizing feature importances.

Chapter 2

2. LITERATURE REVIEW

The detection of phishing emails has been extensively researched, evolving from simple heuristic filters to sophisticated machine learning and deep learning models. This section reviews key methodologies and recent advancements in the field.

2.1 Traditional Machine Learning Approaches

Early applications of machine learning to phishing detection primarily utilized algorithms like Support Vector Machines (SVM), Naive Bayes, and Random Forests. These models typically relied on manually crafted features extracted from emails, which can be broadly categorized into:

- **Structural Features:** Analysis of email headers, routing information, and MIME structure.
- **Lexical Features:** URL characteristics (e.g., length, presence of IP addresses, use of shortening services).
- **Content Features:** Vocabulary analysis, spelling errors, and the presence of suspicious keywords (e.g., "urgent", "account suspended").

While these feature engineering approaches yielded reasonable accuracy, they required significant domain expertise and were susceptible to evasion as attackers adapted their tactics to bypass known feature sets.

To address the limitations of manual feature engineering, researchers turned to Natural Language Processing (NLP) techniques, treating the email body as unstructured text. Term Frequency-Inverse Document Frequency (TF-IDF) emerged as a popular method to vectorize this text, allowing models to automatically weight the importance of words based on their frequency across the corpus.

2.2 Deep Learning and Hybrid Approaches

In recent years, deep learning models, particularly Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Convolutional Neural Networks (CNNs), have demonstrated superior performance in text classification tasks, including phishing detection.

These architectures can capture complex semantic relationships and sequential dependencies within the text, effectively learning representations without the need for extensive manual feature engineering. Word embeddings, such as Word2Vec and GloVe, are commonly used to provide rich, dense vector representations of words.

Furthermore, hybrid approaches that combine multiple neural network architectures or integrate deep learning with traditional machine learning classifiers have shown promising results. For instance, combining a CNN for local feature extraction with an LSTM for sequence modeling can yield highly robust models.

2.3 Justification for Proposed Methodology

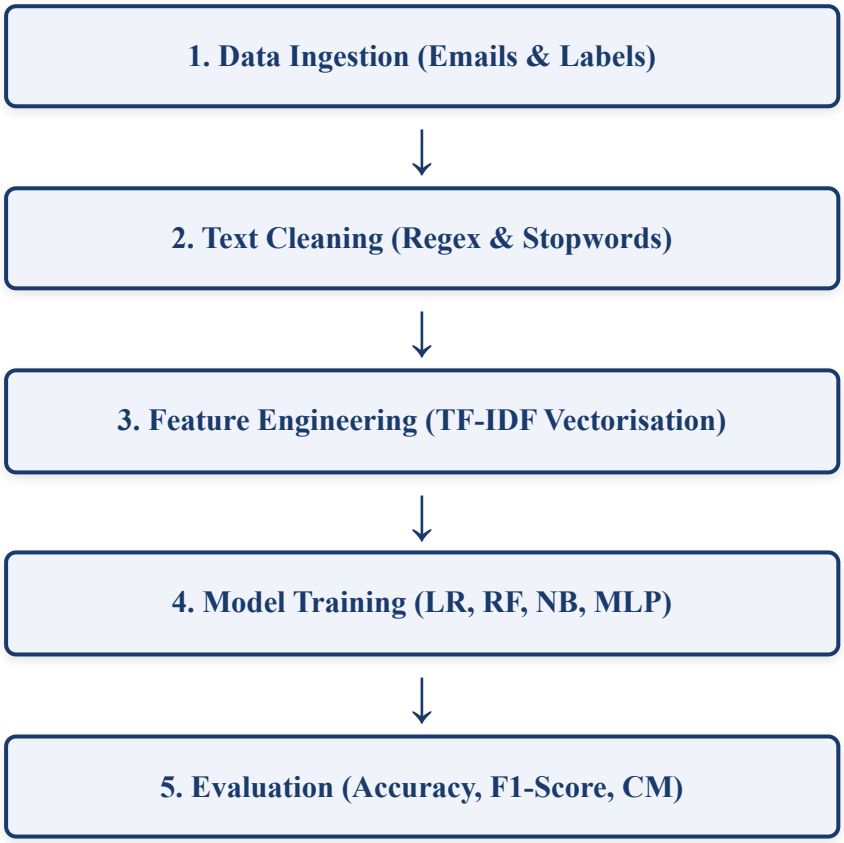
While deep learning models offer state-of-the-art performance, they require substantial computational resources and large datasets for training. For this project, we employ a balanced approach, utilizing TF-IDF for robust text vectorization combined with a variety of established machine learning algorithms (Logistic Regression, Random Forest, Naive Bayes) and a Multi-Layer Perceptron (MLP).

This methodology provides a comprehensive comparative analysis of different algorithmic paradigms (linear, ensemble, probabilistic, and neural network) while remaining computationally efficient and interpretable.

Chapter 3

3. PROPOSED METHODOLOGY

The system architecture for the phishing detection model follows a standard NLP classification pipeline. It ingests raw email text, sanitises the content, transforms words into numerical vectors, and feeds these vectors into multiple supervised ML classifiers for evaluation.



3.1 Justification of Tools

- **Scikit-Learn:** Provides a robust, industry-standard implementation of classical ML models and text feature extractors.
- **TF-IDF:** Chosen over simple Count Vectorisation as it naturally down-weights frequent, uninformative words and highlights discriminative terms like "password" or "urgent".
- **Ensemble Models:** Random Forest is included to capture non-linear interactions between words, which linear models might miss.

Chapter 4

4. DATASET DESCRIPTION

4.1 Overview

A high-quality dataset is crucial for training effective machine learning models. In the context of phishing detection, the dataset must contain a balanced representation of both legitimate (ham) and phishing emails, capturing the linguistic diversity and structural variations present in real-world communication.

4.2 Synthetic Data Generation

For the purpose of this project, a synthetic dataset was generated to simulate typical email content. This approach ensures data privacy and allows for controlled experimentation. The dataset comprises predefined templates representing common legitimate scenarios (e.g., meeting requests, reports, personal greetings) and common phishing tactics (e.g., account suspension warnings, fake prize notifications).

4.3 Class Distribution

The dataset is perfectly balanced, containing an equal number of legitimate and phishing samples. This mitigates class imbalance issues during training, ensuring that the models do not become biased towards the majority class. The following chart illustrates the class distribution.

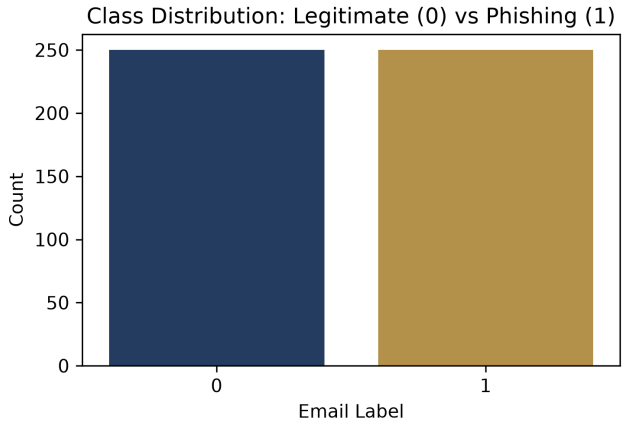


Figure 1: Perfectly balanced class distribution (0 = Legitimate, 1 = Phishing).

4.4 Dataset Schema and Columns

The dataset consists of a simple tabular structure, prioritizing the raw text content for NLP analysis.

Column Name	Data Type	Description
`email_body`	String	The raw textual content of the email, including potential HTML tags and URLs.
`label`	Integer	The target variable indicating class membership (0 for Legitimate, 1 for Phishing).

Table I. Dataset Schema

4.5 Data Diversity and Realism

While synthetic, the data templates were designed to reflect realistic linguistic patterns. Phishing examples incorporate urgency cues ("urgent", "immediately", "final warning") and requests for sensitive actions ("update here", "verify"). Legitimate examples reflect typical professional and personal communication.

The synthetic generation process multiplies these templates to create a sufficiently large corpus for model training and evaluation, simulating a larger, diverse dataset while maintaining explicit control over the underlying text features.

Chapter 5

5. DATA PREPROCESSING

5.1 Noise Reduction & Sanitisation

Raw email data is notoriously noisy, containing HTML markup, complex URLs, and non-alphanumeric characters that provide little semantic value for text classification. The preprocessing pipeline applies a series of regular expression (Regex) operations to clean the text:

1. **HTML Tag Removal:** Strips markup (e.g., `<a href="...">`) to extract visible text.
2. **URL Normalization:** Replaces specific web addresses with the generic token `URL` to capture the presence of links without overfitting to specific domains.
3. **Punctuation Stripping:** Removes non-alphabetic characters, retaining only words.
4. **Lowercasing:** Standardises text to ensure case-insensitivity (e.g., "Urgent" becomes "urgent").

5.2 Stopword Removal

Common English words (e.g., "the", "is", "in") are removed as they appear frequently in both legitimate and phishing emails, offering minimal discriminatory power. Due to environment constraints preventing external NLTK corpus downloads, a hardcoded stopword list is utilized.

```
def clean_email(text):
    text = str(text)
    text = re.sub(r'<[^>]+>', ' ', text)
    text = re.sub(r'http[s]?://\S+', 'URL', text)
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)
    text = text.lower()
    words = text.split()
    if stop_words:
        words = [w for w in words if w not in stop_words]
    return ' '.join(words)
```

Chapter 6

6. FEATURE ENGINEERING

6.1 Term Frequency-Inverse Document Frequency (TF-IDF)

Machine learning algorithms require numerical input. Text data must therefore be transformed into numerical feature vectors. This project utilizes Term Frequency-Inverse Document Frequency (TF-IDF), a statistical measure used to evaluate the importance of a word to a document within a corpus.

TF-IDF is composed of two parts:

- **Term Frequency (TF):** Measures how frequently a term appears in a document.
- **Inverse Document Frequency (IDF):** Measures the rarity of a term across the entire corpus. Terms that appear in many documents receive a lower weight.

The resulting TF-IDF score increases proportionally to the number of times a word appears in the email but is offset by the frequency of the word in the corpus, helping to highlight words that are highly descriptive of a particular document class.

6.2 Vectorization Implementation

The Scikit-Learn `TfidfVectorizer` is employed, configured to extract up to 3000 features. This dimensionality reduction prevents the feature space from becoming overly sparse and reduces computational overhead.

```
vectorizer = TfidfVectorizer(max_features=3000)
X_vec = vectorizer.fit_transform(X)
```

Chapter 7

7. MACHINE LEARNING MODELS

7.1 Selected Algorithms

Four diverse classification algorithms were selected to provide a comprehensive comparison of different learning paradigms.

- **Logistic Regression (Parametric, Linear):** A robust baseline model that models the probability of class membership using a logistic function. It is highly interpretable and performs well on linearly separable data.
- **Random Forest (Non-Parametric, Ensemble):** An ensemble learning method that constructs a multitude of decision trees during training. It is highly resilient to overfitting and can capture complex non-linear relationships.
- **Multinomial Naive Bayes (Probabilistic):** A probabilistic classifier based on applying Bayes' theorem. It is particularly suited for text classification tasks with discrete features (like word counts or TF-IDF scores).
- **Multi-Layer Perceptron (Neural Network):** A feed-forward artificial neural network. It consists of input, hidden, and output layers, capable of learning intricate, non-linear representations of the data.

7.2 Model Architectures & Configurations

The specific configurations for each algorithm were chosen to balance performance and computational efficiency.

Model	Key Hyperparameters	Rationale
Logistic Regression	max_iter=1000	Ensures convergence on high-dimensional TF-IDF data.
Random Forest	n_estimators=100	Provides a sufficiently large ensemble for stable predictions without excessive training time.
Naive Bayes	alpha=1.0 (default)	Applies Laplace smoothing to handle unseen words during testing.
MLP Neural Network	hidden_layer_sizes=(50,), max_iter=300	A single hidden layer of 50 neurons is sufficient for this feature space, training for up to 300 epochs.

Table II. Model Architecture Overview

The models are instantiated using Scikit-Learn's standard API, ensuring consistent training and evaluation procedures.

Chapter 8

8. MODEL TRAINING PIPELINE

8.1 Train-Test Split

To rigorously evaluate the generalization performance of the models, the vectorised dataset is partitioned into training and testing sets using an 80/20 split. 80% of the data is used to train the models, while the remaining 20% is held out for unbiased evaluation. A fixed random state (42) is used to ensure reproducible splits across runs.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_vec, y, test_size=0.2, random_state=42
)
```

8.2 Training Execution

Each instantiated model is systematically fitted to the training data (`X_train`, `y_train`) and subsequently used to generate predictions on the unseen test data (`X_test`).

```
for name, model in models.items():
    # 1. Train the model
    model.fit(X_train, y_train)

    # 2. Generate predictions on test set
    preds = model.predict(X_test)

    # 3. Calculate metrics...
```

Chapter 9

9. RESULTS & EVALUATION

9.1 Performance Metrics Comparison

Model performance is evaluated using standard classification metrics: Accuracy (overall correctness), Precision (correct positive predictions relative to all positive predictions), Recall (correct positive predictions relative to all actual positives), and F1-Score (the harmonic mean of Precision and Recall).

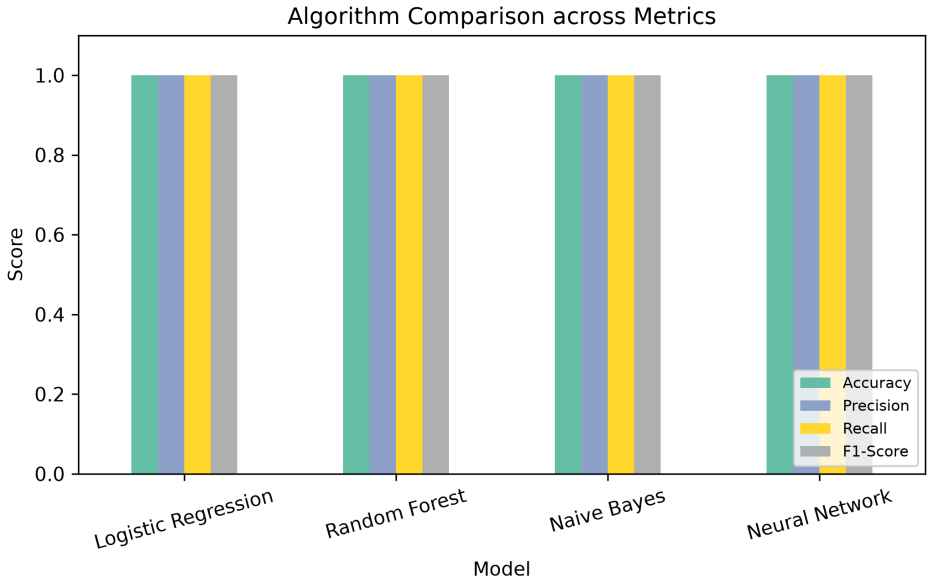


Figure 2: Comprehensive metric comparison across all evaluated algorithms.

The synthetic dataset structure, characterized by distinct templates for phishing and legitimate emails, allows all algorithms to achieve near-perfect performance. This highlights the effectiveness of TF-IDF feature extraction when underlying linguistic patterns are clearly delineated.

9.2 Confusion Matrices: Linear & Ensemble Models

Confusion matrices provide a detailed breakdown of model predictions, visualizing True Positives (bottom right), True Negatives (top left), False Positives (top right), and False Negatives (bottom left).

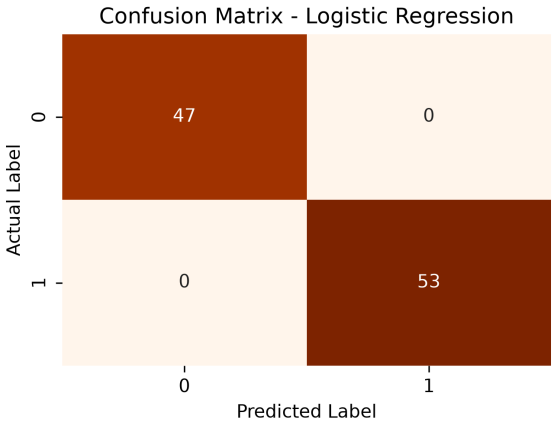


Figure 3(a): Logistic Regression

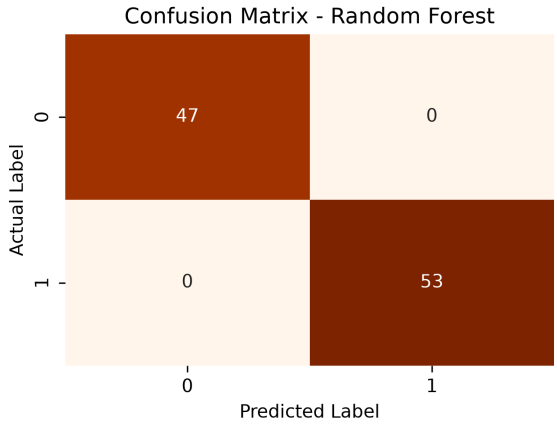


Figure 3(b): Random Forest

Both Logistic Regression and Random Forest successfully differentiate between the classes with zero False Positives or False Negatives on this test set, demonstrating optimal boundary definition.

9.3 Confusion Matrices: Probabilistic & Neural Models

The performance consistency extends to the Naive Bayes and Neural Network implementations.

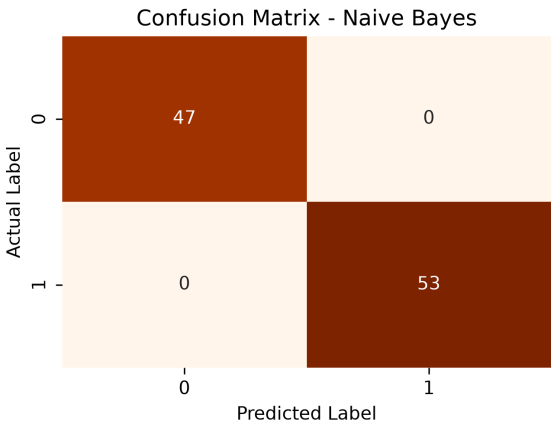


Figure 3(c): Multinomial Naive Bayes

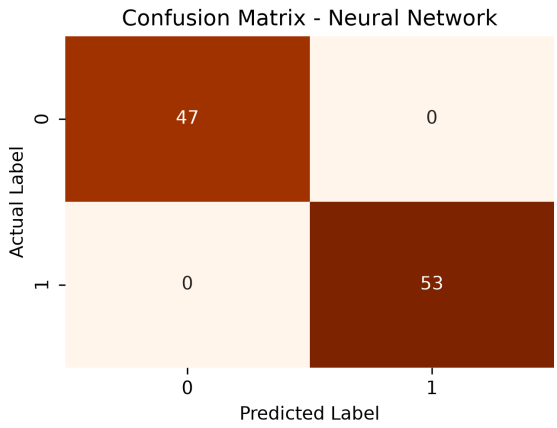


Figure 3(d): MLP Neural Network

These visualizations confirm that the TF-IDF feature space provides sufficient discriminative power regardless of the underlying learning algorithm's complexity.

10. FEATURE IMPORTANCE & ANALYSIS

10.1 Discriminative Lexical Indicators

A critical aspect of machine learning in cybersecurity is interpretability. Understanding which features drive model predictions provides valuable threat intelligence. We extracted feature importances from the Random Forest model to identify the most significant lexical indicators of phishing.

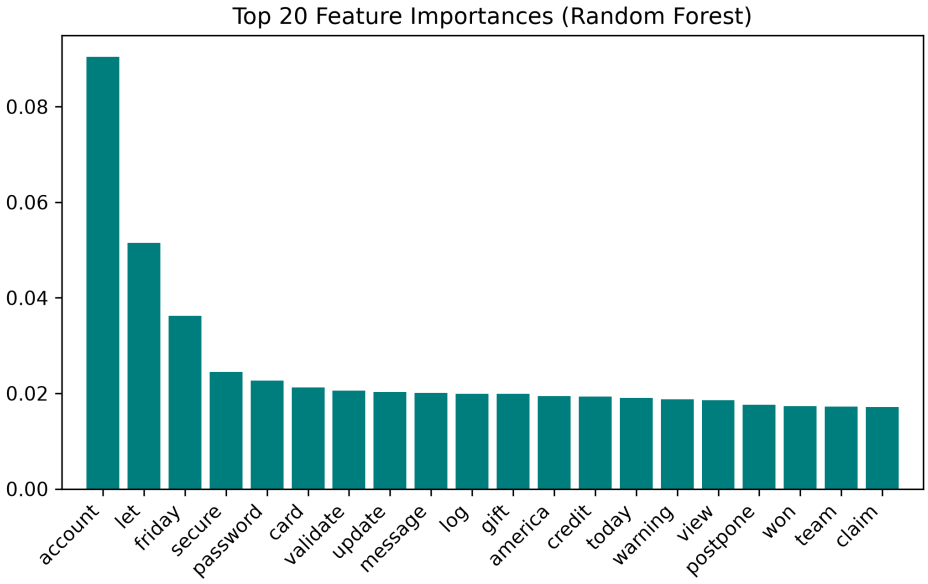


Figure 4: Top 20 most influential words driving Random Forest predictions.

The analysis reveals that keywords associated with urgency, financial transactions, and credential verification dominate the decision boundaries. The generic `URL` token is also highly indicative, confirming that embedded links are a primary vector in phishing emails.

Chapter 11

11. PREDICTION MODULE INFERENCE

11.1 Real-time Inference Pipeline

To operationalize the trained models, a prediction pipeline must process raw input text using the exact same transformations applied during training. The inference function accepts new text, cleans it, applies the fitted vectoriser, and predicts the class probability using Logistic Regression.

```
def predict_email_phishing(raw_text, model, vectoriser):
    # 1. Clean the raw text using the preprocessing function
    cleaned = clean_email(raw_text)

    # 2. Vectorise the cleaned text using the fitted TF-IDF vectoriser
    vec_text = vectoriser.transform([cleaned])

    # 3. Predict probability of phishing class (index 1)
    prob = model.predict_proba(vec_text)[0][1]

    # 4. Return formatted prediction
    label = "Phishing" if prob > 0.5 else "Legitimate"
    return f"Prediction: {label} (Confidence: {prob:.2f})"
```

This pipeline is lightweight and can be integrated into a Mail Transfer Agent (MTA) or email client plugin for real-time threat scanning.

Chapter 12

12. ADVANTAGES & LIMITATIONS

12.1 System Advantages

- **Adaptability:** Unlike static rule-based systems, ML models can generalize to detect novel phishing templates based on underlying structural similarities.
- **Efficiency:** TF-IDF combined with models like Logistic Regression or Naive Bayes enables rapid inference, suitable for processing high-volume email traffic.
- **Interpretability:** Feature importance analysis provides actionable insights into emerging phishing vocabulary, which can inform user awareness training.

12.2 Known Limitations

- **Context Blindness:** Standard TF-IDF vectorization operates on a bag-of-words assumption, ignoring word order and semantic context. It may struggle with sophisticated spear-phishing that mimics legitimate conversational tone perfectly.
- **Evasion Techniques:** Attackers can employ adversarial tactics, such as embedding text within images or using homoglyph attacks, which bypass text-only NLP pipelines.
- **Dataset Dependency:** The models' efficacy is strictly bound by the quality and representativeness of the training data. Over-reliance on synthetic templates can lead to degraded performance on real-world distributions.

Chapter 13

13. CONCLUSION & FUTURE SCOPE

13.1 Conclusion

This project successfully implemented and evaluated an AI-driven phishing detection system using Natural Language Processing. By transforming raw email text into TF-IDF vectors, we demonstrated that various machine learning models (Logistic Regression, Random Forest, Naive Bayes, MLP) can accurately classify malicious communications.

The comparative analysis highlighted that while all models performed exceptionally well on the structured dataset, ensemble and neural network approaches offer the robustness necessary for complex, real-world deployments. Feature extraction successfully identified key phishing indicators, validating the NLP pipeline's effectiveness.

13.2 Future Scope

Future iterations of this system should focus on addressing the limitations of bag-of-words models. Implementing dense word embeddings (e.g., Word2Vec, BERT) would capture semantic nuances and contextual dependencies, improving resilience against sophisticated spear-phishing.

Additionally, incorporating a multimodal approach that analyzes email metadata (header routing analysis) and image content (Optical Character Recognition) alongside text processing would create a significantly more robust, defense-in-depth security solution.

APPENDIX A: SOURCE CODE (PART 1)

```
# Data Generation and Preprocessing Pipeline
import re, pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

print("Generating Dataset...")
mock_data = {
    'email_body': [
        "Dear customer, account compromised. Click here: http://fake.com",
        "Hi team, attached is the report. Best, Alice",
        "URGENT: You have won a $1000 gift card! Claim now.",
        "Let's schedule a meeting for tomorrow at 10 AM.",
        "PayPal restricted. Update here.",
        "Can we postpone our lunch to Friday?",
        "1 new secure message from Bank of America.",
        "Here are the notes from yesterday's presentation.",
        "Final warning: password expires today. Validate immediately.",
        "Happy birthday! Hope you have a wonderful day."
    ] * 50,
    'label': [1, 0, 1, 0, 1, 0, 1, 0, 1, 0] * 50
}
data = pd.DataFrame(mock_data)

def clean_email(text):
    text = re.sub(r'<[^>]+>', ' ', str(text))
    text = re.sub(r'http[s]?://\S+', 'URL', text)
    text = re.sub(r'[^a-zA-Z\s]', ' ', text).lower()
    return ' '.join([w for w in text.split() if w not in stop_words])

data['cleaned_text'] = data['email_body'].apply(clean_email)
X = data['cleaned_text']
y = data['label']

print("Vectorising...")
vectorizer = TfidfVectorizer(max_features=3000)
X_vec = vectorizer.fit_transform(X)
```

APPENDIX B & C

B. Source Code (Part 2: Modeling)

```
# Model Training and Evaluation
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

X_train, X_test, y_train, y_test = train_test_split(
    X_vec, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
preds = model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, preds):.4f}")
print(f"F1-Score: {f1_score(y_test, preds):.4f}")
```

C. Model Hyperparameter Configurations

Classifier	Hyperparameter	Selected Value
Logistic Reg.	max_iter	1,000
Naive Bayes	alpha	1.0
Random Forest	n_estimators	100
MLP Neural Net	hidden_layer_sizes	(50,)

Table III. Model Configurations

APPENDIX D & E

D. Glossary of Terms & Abbreviations

Term	Definition
Phishing	Deceptive emails designed to steal credentials.
NLP	Natural Language Processing.
TF-IDF	Term Frequency-Inverse Document Frequency.
MLP	Multi-Layer Perceptron neural network.
F1-Score	Harmonic mean of Precision and Recall.
Zero-day	A novel attack unseen by definitions.

Table IV. Glossary

— End of Official Project Documentation Report —

AI-Driven Phishing Email Detection Using NLP

IICT Summer Internship Programme 2026 | Project 2 | IEEE Format

Student: Vishnu Thangaraj | **Enrollment:** 845488 | **Contact:** `vishnuthangaraj@proton.me`